

ArogyaMitra

AI-Driven Workout Planning, Nutrition Guidance and Health
Coaching Platform

System Architecture, Design and Implementation Report

Done By: Charan

Mani Sumanth

Pranay Bhaskar

Date of Submission: 29 July 2026

Document Scope

This document describes the complete technical design of ArogyaMitra, an AI powered fitness and wellness platform. It covers the problem that the project addresses, the objectives that were set before development, the system architecture, the technology stack, the database design, and every important module of the backend and frontend. The document also explains how the artificial intelligence layer works internally, how the AROMI coach responds to user messages, and how third party services such as Groq, YouTube, Spoonacular and Google Calendar are integrated into the platform.

Purpose

The purpose of this document is to provide a single, organised reference that explains why ArogyaMitra was built, how it is structured internally, and how each component of the system communicates with the others. It is intended to help a reader understand the platform without needing to read the source code directly, and it can also be used as a reference during future maintenance, testing or extension of the project.

Intended Audience

This document is written for developers who may work on the project in the future, for reviewers and mentors who are evaluating the project, and for any technically inclined reader who wants to understand how a modern AI powered health platform is designed from the ground up. Basic familiarity with web development concepts such as APIs, databases and frontend frameworks is assumed.

Chapter 1: Project Overview

ArogyaMitra is an AI powered fitness and wellness platform that has been designed to provide personalised health guidance to its users through intelligent workout planning, nutrition recommendations and real time wellness support. The name ArogyaMitra itself comes from two Sanskrit derived words, where 'Arogya' means good health and 'Mitra' means friend, which together represent the core idea of the project — a friendly companion that guides a person towards a healthier lifestyle.

The platform has been built using a modern and scalable technology stack. FastAPI is used to build the backend, React is used to build the frontend interface, and Groq's LLaMA-3.3-70B Versatile model is used as the intelligence layer that generates human like coaching responses and customised fitness plans. The system takes into account each individual's lifestyle, preferences and health profile before generating any recommendation, which makes every plan feel personal rather than generic.

At the centre of the platform is AROMI, an interactive AI coach that a user can talk to at any point in their journey. AROMI is capable of understanding natural language messages such as travel plans, injuries, mood changes or time constraints, and it adjusts the workout and nutrition plan of the user accordingly, in real time, without requiring the user to manually edit anything.

The platform also introduces a unique gamification element in the form of charity contributions that are linked to fitness achievements. As users complete workouts, track meals and burn calories, a small notional donation amount accumulates on their dashboard, which is intended to motivate consistency by connecting personal health with a sense of social contribution.

Chapter 2: Problem Statement

Most people who want to improve their health and fitness face the same recurring set of problems. Generic workout plans found on the internet do not consider a person's medical history, injuries, allergies or available time, which often leads to poor results or, in some cases, injury. Nutrition advice is usually not localised, and a person following an Indian diet often finds it difficult to translate western meal plans into food that is actually available and familiar to them.

Another major problem is the lack of adaptability. A fitness plan created once, whether by a trainer or an application, usually remains static. If a person travels for a few days, gets injured, or simply feels low on energy, most existing applications have no mechanism to intelligently adjust the plan. The user is left to either abandon the plan altogether or manually search for alternatives, which reduces long term consistency.

Motivation is also a well documented problem in the fitness domain. Many users start a fitness journey with enthusiasm but lose interest within a few weeks because there is no emotional or social reward attached to their progress. Existing applications rely mostly on streak counters and badges, which quickly lose their appeal.

ArogyaMitra was conceived to directly address these three problems together — personalisation, adaptability and motivation — by combining a large language model with structured health data, external content sources and a charity based reward system.

Chapter 3: Objectives

The following objectives were defined at the start of the project and guided every design decision that followed.

To design a platform that generates a personalised seven day workout routine based on a user's fitness level, goal, available time and workout preference.

To design an intelligent nutrition module that respects dietary restrictions, allergies and calorie targets while keeping meals rooted in Indian cuisine.

To build AROMI, a conversational AI coach that can dynamically modify workout and nutrition plans in response to natural language input from the user.

To integrate trustworthy third party data sources, namely YouTube for exercise demonstration videos and Spoonacular for nutritional and recipe data.

To allow a user's fitness schedule to sync automatically with their personal Google Calendar so that workouts and meals appear as reminders in daily life.

To track a user's progress over time through visual analytics covering workouts completed, calories burned, weight change and body metrics.

To introduce a charity gamification system that converts fitness activity into a notional donation amount, encouraging users to remain consistent.

To build the entire system using a modular and secure architecture that can be extended easily with future features such as wearable integration.

Chapter 4: System Requirements

4.1 Functional Requirements

Functional requirements describe what the system must actually do from the point of view of the user and the platform.

The system shall allow a new user to register with basic personal and fitness related information.

The system shall allow a registered user to securely log in using a username or email along with a password.

The system shall collect a detailed health assessment consisting of fourteen questions covering age, gender, body metrics, fitness level, goal, workout preference, medical history, injuries, allergies and medication.

The system shall generate a seven day workout plan using the Groq LLaMA-3.3-70B model based on the collected health assessment.

The system shall attach a relevant YouTube demonstration video to every exercise inside the generated workout plan.

The system shall generate a seven day nutrition plan that includes breakfast, lunch, dinner and snacks, along with an automatically generated grocery shopping list.

The system shall allow the user to converse with AROMI in natural language and receive contextually adjusted recommendations.

The system shall allow the user to optionally sync their generated plans with their Google Calendar.

The system shall track completed workouts, completed meals and body metrics over time and display this information as charts on a progress page.

The system shall calculate a notional charity contribution amount based on the user's workouts completed, calories burned and meals tracked.

4.2 Non Functional Requirements

Non functional requirements describe the qualities the system must possess rather than the specific actions it performs.

Performance: the backend should respond to standard API requests within an acceptable time, and AI generated responses should typically return within a few seconds due to the high inference speed of the Groq platform.

Security: passwords must be hashed before storage, and all protected routes must require a valid JSON Web Token.

Scalability: the backend must be modular, using separate routers and services for each feature so that new modules can be added without disturbing existing ones.

Usability: the frontend interface must be responsive and usable on both desktop and mobile screen sizes.

Maintainability: the codebase must follow a clean separation between routers, models, schemas and services to make debugging and future development easier.

Availability: the system should degrade gracefully if an external API such as Spoonacular or YouTube is temporarily unavailable, rather than crashing the entire request.

4.3 User Requirements

The intended user of ArogyaMitra is any individual who wants a personalised and adaptable approach to fitness, ranging from a complete beginner exploring home workouts to a more experienced user following a structured strength training programme. The user is expected to have basic comfort with using a web or mobile browser, and no prior technical knowledge is required to operate the platform, since every module has been designed with a simple, guided, conversational interface.

Chapter 5: System Architecture

The overall architecture of ArogyaMitra follows a layered design in which the React frontend communicates with the FastAPI backend through REST APIs, and the backend in turn communicates with a set of AI agents, a relational database and several external services. This separation of concerns keeps each layer independent, testable and replaceable.

5.1 Backend Architecture

The backend is built entirely on FastAPI and is organised into modular routers, each responsible for one feature area such as authentication, workouts, nutrition, progress tracking, health assessment, the AROMI coach, and administration. Each router calls into a corresponding service layer that contains the actual business logic, keeping the routing layer thin and focused only on request and response handling.

5.2 Frontend Architecture

The frontend is a single page application built with React and Vite, styled using Tailwind CSS and animated using Framer Motion. It communicates with the backend exclusively through Axios based API calls and manages global state using Zustand, which keeps user authentication, workout data and chat history synchronised across every page without unnecessary re rendering.

5.3 AI Layer

The AI layer is powered by the Groq LLaMA-3.3-70B Versatile model. This layer receives structured user data, such as a health assessment or a chat message, converts it into a carefully engineered prompt, and returns a structured response that the backend can parse and store. The same underlying model powers workout generation, nutrition generation and the AROMI conversational coach.

5.4 Database Layer

Data is stored using SQLAlchemy models on top of a SQLite database for local development, with the system designed so that a production deployment could move to a more scalable relational database without changing the application logic. Core tables include Users, WorkoutPlan, Exercise, NutritionPlan, Meal, HealthAssessment, ProgressRecord and ChatSession.

5.5 Authentication

Authentication is implemented using JSON Web Tokens. When a user registers or logs in, the backend issues a signed token that the frontend stores and attaches to every subsequent request. Protected routes verify this token before allowing access to personalised data such as workout plans or chat history.

5.6 Calendar Integration

Google Calendar integration allows a user's generated workout and nutrition schedule to be pushed directly into their personal calendar as events, complete with reminders. This closes the gap between having a plan and actually remembering to follow it.

5.7 Groq Integration

The Groq client is initialised once when the backend starts and is reused across all AI powered features. Because Groq is known for its very fast inference speed compared to many other large language model providers, the platform is able to deliver near real time responses even for relatively long generation tasks such as a complete seven day workout plan.

Chapter 6: Technology Stack

The technology stack for ArogyaMitra was chosen carefully to balance developer productivity, performance and long term maintainability.

Technology

Role in the Project

React

Builds the entire interactive frontend user interface

FastAPI

Serves as the backend web framework and REST API layer

Python

The core programming language used across the backend

SQLAlchemy

Object relational mapper used to define and query database models

SQLite

Lightweight relational database used for local development and storage

Groq API (LLaMA-3.3-70B)

Provides the large language model used for AI generated plans and coaching

Axios

Handles all HTTP communication between the frontend and backend

React Router

Manages page navigation and routing inside the frontend application

Tailwind CSS

Provides utility based styling for a clean and responsive interface

JWT (JSON Web Token)

Provides secure, stateless authentication between client and server

In addition to the technologies listed above, the project also relies on Zustand for lightweight state management, Framer Motion for smooth page and component animations, and python-dotenv for safely loading environment variables such as API keys without exposing them inside the source code.

Chapter 7: Database Design

7.1 ER Diagram Overview

The database has been designed around the central User entity, which is connected to every other important entity in the system, either directly or indirectly. A user has one or more workout plans, one or more nutrition plans, one health assessment, several progress records and a running history of chat sessions with AROMI.

7.2 Table Design

Table

Key Fields

Purpose

User

id, username, email, password_hash, fitness_goal, workout_preference, diet_preference

Stores account and profile information

HealthAssessment

id, user_id, medical_history, injuries, allergies, medications

Stores health related answers collected during onboarding

WorkoutPlan

id, user_id, day, focus_area, duration, status

Stores each day's generated workout plan

Exercise

id, workout_plan_id, name, sets, reps, rest, video_url

Stores individual exercises inside a workout plan

NutritionPlan

id, user_id, day, calories, diet_type

Stores each day's generated nutrition plan

Meal

id, nutrition_plan_id, meal_type, name, calories, ingredients

Stores individual meals inside a nutrition plan

ProgressRecord

id, user_id, date, calories_burned, weight, workouts_done

Stores daily and weekly progress analytics

ChatSession

id, user_id, message, sender, timestamp

Stores the conversation history with AROMI

7.3 Relationships

The User table acts as the parent of almost every other table through a one to many relationship. One user can have many workout plans across different days, and each workout plan in turn contains many exercises. The same pattern is repeated for nutrition plans and meals. Progress records and chat sessions are also linked back to the user through a foreign key, which allows the system to retrieve a complete history for any given account.

7.4 Indexes

Primary keys are indexed automatically, and additional indexes are maintained on frequently queried foreign key columns such as user_id across the WorkoutPlan, NutritionPlan, ProgressRecord and ChatSession tables. This keeps lookups fast even as the chat history and progress records grow over time.

7.5 Constraints

Email and username fields on the User table carry a uniqueness constraint to prevent duplicate accounts. Foreign key constraints ensure that an exercise cannot exist without a valid parent workout plan, and that a meal cannot exist without a valid parent nutrition plan, which keeps the database in a consistent state at all times.

Chapter 8: Backend Architecture

8.1 Routers

The backend is divided into modular routers, each mounted on its own path. The Auth router handles registration and login. The Users router manages profile data. The Workouts router manages plan generation and retrieval. The Nutrition router manages meal plan generation. The Progress router manages analytics. The Admin router manages administrative operations. The Chat router and the Health Assessment router manage the AROMI conversation and the onboarding questionnaire respectively, while the AROMI Coach router specifically handles adaptive plan modification requests such as travel or injury adjustments.

8.2 Models

Database models are defined using SQLAlchemy and make heavy use of Python enumerations to keep data consistent. For example, FitnessGoal is defined as an enumeration with values such as weight loss, weight gain, muscle gain, maintenance and endurance, while WorkoutPreference covers home, gym, outdoor and hybrid options, and DietPreference covers vegetarian, non vegetarian, vegan, keto and paleo diets.

8.3 Schemas

Pydantic models are used as request and response schemas throughout the backend. The UserRegister schema, for instance, validates the incoming registration payload and enforces correct types for fields such as email, age, height, weight, fitness level, fitness goal, workout preference and diet preference before the data ever reaches the database layer.

8.4 Services

The service layer contains the actual business logic of the platform, separated cleanly from the routing layer. Services exist for workout generation, nutrition planning, calorie estimation, analytics processing and dynamic plan adjustment. This layer is also responsible for orchestrating calls to external APIs including Groq, YouTube, Spoonacular and Google Calendar.

8.5 Prompt Engineering

A significant part of the backend logic involves constructing prompts that are sent to the Groq model. Each prompt is built dynamically using the user's health assessment, current goal, dietary restrictions and any special context such as travel or injury, ensuring that the model always receives complete and relevant information before generating a response.

8.6 Business Logic

Beyond simple AI calls, the backend also applies its own business rules on top of the model's output, such as validating that a generated plan actually respects the user's stated allergies, calculating rest day placement across the week, and computing the notional charity donation amount from completed activity.

Chapter 9: Frontend Architecture

9.1 Pages

The frontend consists of a set of dedicated pages, including the landing page, registration page, login page, dashboard, workout plans page, nutrition plans page, health assessment page, progress tracking page, AI coach page and the profile page. Each page is responsible for one clear part of the user journey.

9.2 Components

Reusable components such as the Navbar, LoadingSpinner, StatCard, BackgroundImage, CharityImpactCard, Achievements and the ArogyaCoach floating widget are shared across multiple pages, which keeps the codebase consistent and avoids duplicated styling logic.

9.3 Routing

Page navigation is handled using React Router, which allows the application to behave as a single page application while still supporting clean, bookmarkable URLs for each major section such as the dashboard, workouts and progress pages.

9.4 Reusable Components

Beyond layout components, the Exercise Player is a particularly important reusable component. It embeds a YouTube instructional video alongside a live camera preview, a countdown timer, a sets and reps tracker, and simple activity detection that estimates whether the user is actively moving during the exercise.

9.5 State Management

Zustand is used to manage global application state such as the logged in user's profile, authentication token, and cached workout and nutrition data. Its lightweight design avoids the boilerplate typically associated with larger state management libraries while still preventing unnecessary component re renders.

9.6 API Calls

All communication with the backend is centralised through a small set of API service files built on top of Axios, such as authApi, userApi, workoutApi and chatApi. Centralising these calls makes it straightforward to attach the authentication token automatically and to handle errors consistently across the entire application.

Chapter 10: AI System

10.1 Prompt Engineering

Every AI powered feature in ArogyaMitra begins with a structured prompt. Rather than sending a single generic instruction to the model, the backend assembles a detailed prompt containing the user's fitness goal, fitness level, workout preference, diet preference, medical history, injuries, allergies and medications, so that the model has full context before it generates anything.

10.2 Health Profile

The health profile of a user, collected through the fourteen question assessment, forms the foundation of every AI request. This profile is stored once and reused across workout generation, nutrition generation and AROMI conversations, so the user never has to repeat their health information.

10.3 Workout Generation

When a workout plan is requested, the AI agent sends the user's profile to the Groq model along with an instruction to produce a structured seven day plan containing warm ups, exercises with sets, reps and rest periods, and daily fitness tips. The response is parsed into a structured format before being stored in the WorkoutPlan and Exercise tables.

10.4 Nutrition Generation

Nutrition generation follows a similar pattern but combines the Groq model's reasoning with real nutritional data retrieved from the Spoonacular API, ensuring that calorie counts and macro breakdowns are grounded in actual food data rather than being estimated purely by the language model.

10.5 Intent Classification

When a user sends a message to AROMI, the backend first classifies the intent behind the message, distinguishing between a general question, a request for motivation, and a request that requires an actual change to the workout or nutrition plan, such as travel, injury, or a change in available time.

10.6 Workout Modification

If the intent is classified as a workout modification request, the DynamicPlanAdjustmentRequest schema is populated with the reason, the number

of days affected, the user's current plan and their profile data, and this is sent to the model so that it can produce an adjusted plan that fits the new circumstance.

10.7 Nutrition Modification

Nutrition modification works in the same way as workout modification, allowing AROMI to suggest travel friendly meal alternatives or adjusted portion sizes when a user's routine changes, while still respecting their stated allergies and dietary preference.

10.8 Future Memory System

At present, AROMI relies on the conversation history stored within a session to maintain context. A planned future enhancement is a longer term memory system that would allow AROMI to recall patterns across weeks or months, such as recurring excuses for missed workouts or gradually changing food preferences, and use that history to give more personalised coaching over time.

Chapter 11: Chatbot Workflow

The AROMI chatbot follows a clear and predictable request lifecycle every time a user sends a message. Understanding this lifecycle is useful for anyone debugging the chat feature or extending it with new capabilities.

11.1 Complete Request Lifecycle

The user types a message into the AROMI chat widget on the React frontend.

The React frontend sends the message to the FastAPI backend through the `aromi-chat` endpoint, along with the authentication token.

FastAPI verifies the token and retrieves the current user's profile, including their health assessment and diet preference.

The backend constructs a prompt combining the user's message, their profile, and their current workout and nutrition plan, and sends it to Groq.

Groq's LLaMA-3.3-70B model processes the prompt and returns a natural language response, along with any structured plan adjustments if required.

The backend stores the exchange inside the database as a new chat session entry so that history is preserved for future context.

The response, along with any updated plan data, is sent back to the React frontend.

The frontend renders AROMI's reply inside the chat window and updates the relevant workout or nutrition page if a plan change occurred.

This lifecycle can be summarised as a flow moving from the user, through React, into FastAPI, through profile retrieval, into Groq, back through the database for storage, and finally returning to the frontend for display, which keeps every conversation both responsive and grounded in the user's actual data.

Chapter 12: Workout Generation Engine

12.1 Generation

The workout generation engine begins by reading the user's fitness goal, fitness level, workout preference and available time, and combines this with any medical history or injury information collected during the health assessment. This combined profile is sent to the Groq model with a clear instruction to produce a full seven day schedule.

12.2 Storage

Once a plan is generated, it is broken down and stored across the WorkoutPlan and Exercise tables, with one WorkoutPlan row created for each day of the week and multiple Exercise rows linked to each plan. Rest days are stored explicitly so the frontend can display them clearly rather than leaving a gap in the schedule.

12.3 Modification

A user can request a modification to their workout plan at any time through AROMI, whether due to travel, fatigue or a missed session. The modification service retrieves the existing plan and asks the model to adjust only the affected days, keeping the rest of the week unchanged wherever possible.

12.4 Regeneration

If a user's fundamental profile changes, for example if their fitness goal shifts from weight loss to muscle gain, the system supports a full regeneration of the workout plan rather than a partial modification, ensuring that the entire week remains coherent with the new goal.

12.5 Synchronisation

After generation or modification, the updated schedule can optionally be synchronised to the user's Google Calendar, where each workout appears as an event with its recommended time window, duration and a short description generated by AROMI.

Chapter 13: Nutrition Engine

13.1 Personalisation

The nutrition engine personalises every meal plan around three inputs collected during onboarding: the user's diet preference, such as vegetarian or non vegetarian, any stated food allergies, and a calorie target that is either provided directly by the user or estimated from their body metrics and fitness goal.

13.2 Calorie Estimation

When a specific calorie target is not provided, the engine estimates a suitable daily calorie count using the user's age, gender, height, weight and fitness goal, following standard estimation principles before passing this figure into the meal generation prompt.

13.3 Meal Planning

The Groq model, combined with data retrieved from the Spoonacular API, produces a full seven day plan covering breakfast, lunch, dinner and a snack, deliberately rooted in familiar Indian dishes such as dosa, roti, and dal, so that the recommendations feel practical rather than foreign to the target audience.

13.4 Modification

Just as with workouts, a user can ask AROMI to adjust their meal plan, for instance while travelling, and the engine will substitute easily available local alternatives while still respecting the user's allergy and dietary constraints.

13.5 Storage

Generated meals are stored in the NutritionPlan and Meal tables, and a consolidated weekly shopping list is derived automatically by aggregating the ingredients required across every meal in the plan, which the user can then use directly while shopping online.

Chapter 14: Calendar Integration

14.1 Current System

In its current form, ArogyaMitra allows a user to connect their Google account once from the dashboard, after which any newly generated or modified workout and nutrition plan can be pushed to their calendar with a single confirmation during the health assessment flow.

14.2 Google Calendar

The integration uses the Google Calendar API, authenticated through a client ID, client secret and redirect URI configured in the backend environment file. Once authorised, the backend is able to create, update and remove calendar events on behalf of the user without requiring repeated logins.

14.3 Scheduling

Each workout or meal is translated into a calendar event with an appropriate start time, duration and reminder, based on the time preference the user selected, such as an early morning or evening workout window.

14.4 Future Automation

A planned improvement is automatic weekly resynchronisation, so that a new week's plan is pushed to the calendar without the user needing to revisit the health assessment page every time.

14.5 Synchronisation Logic

The synchronisation logic is designed to be idempotent, meaning that re running the sync process for the same week will update existing events rather than creating duplicate entries, which keeps the user's calendar clean even if a plan is regenerated multiple times.

Chapter 15: REST APIs

The following table summarises the most important REST endpoints exposed by the FastAPI backend.

Endpoint

Method

Purpose

/api/auth/register

POST

Registers a new user account

/api/auth/login

POST

Authenticates a user and returns a JWT token

/api/health-assessment/assessment/submit

POST

Submits the fourteen question health assessment

/api/health-analysis/analyze

POST

Runs the AI powered health analysis on submitted data

/api/workouts/generate

POST

Generates a seven day workout plan for the user

/api/nutrition/generate

POST

Generates a seven day nutrition plan for the user

/api/aromi-chat

POST

Sends a message to AROMI and returns a coaching response

/api/progress

GET

Retrieves progress analytics for the logged in user

/api/calendar/sync

POST

Synchronises the current plan with Google Calendar

/docs

GET

Displays the interactive Swagger documentation for all endpoints

Every protected endpoint expects a Bearer token in the Authorization header. A typical request flow begins with the frontend calling the login endpoint to obtain a token, after which every subsequent call attaches this token automatically through an Axios interceptor, and the backend responds with a standard JSON structure containing either the requested data or a descriptive error message.

Chapter 16: Authentication

16.1 JWT

Authentication in ArogyaMitra is built around JSON Web Tokens. When a user logs in successfully, the backend generates a signed token containing the user's identifier and issues it to the frontend, which stores it and includes it with every future request.

16.2 Login

The login endpoint accepts either a username or an email address along with a password. The backend compares the submitted password against the stored hashed password using a secure hashing algorithm, and only issues a token if the credentials match.

16.3 Registration

The registration endpoint validates a new user's email, username, password and optional profile fields such as age, gender, height and weight using Pydantic models before creating a new record in the database, with the password hashed prior to storage.

16.4 Protected Routes

Almost every route beyond authentication itself is protected. A dependency function checks the incoming token on every request, decodes it, and retrieves the current user, rejecting the request with an appropriate error if the token is missing, expired or invalid.

16.5 Security

In addition to password hashing and JWT verification, the backend stores sensitive credentials such as the Groq API key, YouTube API key, Spoonacular key and Google Calendar credentials inside environment variables rather than inside the source code, and Cross Origin Resource Sharing is restricted to the known frontend origins during development.

Chapter 17: User Interface

17.1 Design Philosophy

The interface of ArogyaMitra follows a calm, modern and slightly playful design philosophy, aiming to feel encouraging rather than clinical, which is important for a platform dealing with personal health information.

17.2 Theme

Both a dark mode and a light mode are supported across the dashboard and every internal page, allowing the user to choose the appearance that is most comfortable for them at any time of day.

17.3 Cards

Information throughout the dashboard is organised into clearly separated cards, such as the Quick Actions card, the Charity Impact card, and the Today's Remaining Tasks card, which keeps related information grouped together visually.

17.4 Sidebar and Navigation

A persistent top navigation bar provides quick access to the Dashboard, Workouts, Nutrition, Progress and AI Coach pages from anywhere within the application, keeping navigation consistent across the entire user journey.

17.5 Minimalism

The visual design intentionally avoids clutter, using generous spacing and a limited colour palette centred around purple and blue gradients, which keeps the focus on the actual content rather than on decorative elements.

17.6 Typography and Spacing

A clean sans serif typeface is used consistently across headings and body text, with clear size differences between page titles, card titles and descriptive text, making the hierarchy of information easy to scan quickly.

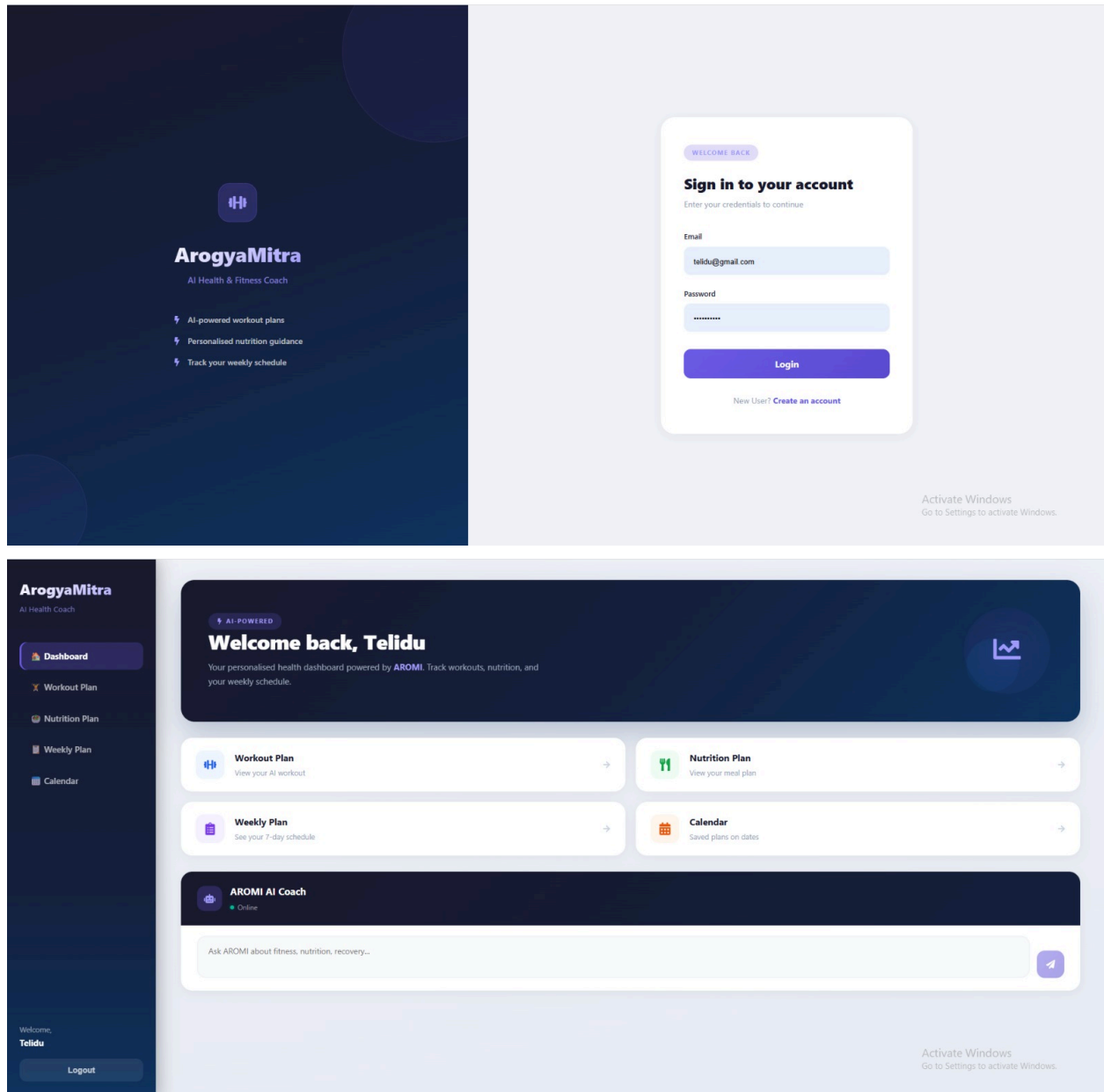
17.7 Animations

Framer Motion is used to add subtle animations, such as fading transitions when a page loads or a card appears, which makes the interface feel responsive without being distracting.

17.8 Accessibility

Interactive elements such as buttons and form fields are given clear labels and sufficient colour contrast, and the layout remains fully responsive across different screen sizes so the platform can be comfortably used on both desktop and mobile devices.

18. Output Screens



ArogyaMitra

AI Health Coach

Dashboard

Workout Plan

Nutrition Plan

Weekly Plan

Calendar

Welcome,

Ryan Gosling

Logout

AI-GENERATED

Today's Workout

Personalised by AROMI based on your health profile. Navigate between days to see each session.

5

Days Total

Thursday

Today's Focus

1

~60 min

Per Session

<

Thu

Fri

Sat

Sun

Mon

>

Day 1 of 5

Thursday

Day 1: Chest and Inceps

Warm-up (10 minutes)

+ 5 minutes on the treadmill (jogging)

+ Dynamic stretching (arm circles, leg swings, torso twists)

Main Exercises

1. Barbell Bench Press (3 sets of 8-12 reps)

2. Incline Dumbbell Press (3 sets of 10-12 reps)

3. Cable Fly (3 sets of 12-15 reps)

4. Tricep Pushdown (3 sets of 10-12 reps)

5. Overhead Dumbbell Extension (3 sets of 12-15 reps)

Cool-down (10 minutes)

Progress Tracker

25%

2 / 8 exercises completed

CALORIES BURNED

95 / 340 kcal

+ 5 minutes on the treadmill

Activate Windows

Go to Settings to activate Windows.

+ Dynamic stretching (arm circles, leg swings, torso

ArogyaMitra

AI Health Coach

Dashboard

Workout Plan

Nutrition Plan

Weekly Plan

Calendar

Welcome,

Ryan Gosling

Logout

calorie guidance.

5 Days

Meal Plan

Thursday

Today's Meals

4 Meals

Per Day

<

Thu

Fri

Sat

Sun

Mon

>

Day 1 of 5

Thursday

Day 1

Breakfast: Oatmeal with banana, almond milk, and walnuts (350 calories)

+ 1 cup cooked oatmeal

+ 1 medium banana

+ 1 cup almond milk

+ 1 tablespoon chopped walnuts

Lunch: Grilled chicken breast with quinoa, mixed vegetables, and avocado (550 calories)

+ 4 oz grilled chicken breast

+ 1/2 cup cooked quinoa

+ 1 cup mixed vegetables (bell peppers, carrots, broccoli)

+ 1/2 avocado

Dinner: Baked salmon with sweet potato, green beans, and brown rice (500 calories)

+ 4 oz baked salmon

+ 1 medium sweet potato

+ 1 cup green beans

Meal Tracker

33%

1 / 3 meals logged

CALORIES CONSUMED

350 / 1000 kcal

Breakfast

+ 1 cup cooked oatmeal + 1 medium banana + 1 cup almond milk + 1 tablespoon chdp..

Lunch

+ 4 oz grilled chicken breast + 1/2 cup cooked quinoa + 1 cup mixed vegetables + 1/2 avocado

Dinner

+ 4 oz baked salmon + 1 medium sweet potato + 1 cup green beans

ArogyaMitra

AI Health Coach

Dashboard

Workout Plan

Nutrition Plan

Weekly Plan

Calendar

Welcome,

Ryan Gosling

Logout

5-DAY SCHEDULE

Weekly Plan

Your personalised fitness & nutrition schedule by AROMI. Save it to your calendar to stay on track.

5 workout days - 5 meal plans

Save to Calendar

Workout Plan

5-day training schedule

01

Day 1: Chest and Triceps

Warm-up (10 minutes)

Cool-down (10 minutes)

Barbell Bench Press (3 sets of 8)

Incline Dumbbell Press (3 sets of 10)

Cable Fly (3 sets of 12)

Tricep Pushdown (3 sets of 10)

Overhead Dumbbell Extension (3 sets of 12)

WORKOUT

02

Day 2: Back and Biceps

Warm-up (10 minutes)

Cool-down (10 minutes)

Pull-ups (3 sets of 8)

Barbell Rows (3 sets of 8)

Lat Pulldowns (3 sets of 10)

Dumbbell Bicep Curls (3 sets of 10)

Hammer Curls (3 sets of 10)

WORKOUT

03

Day 3: Legs

Warm-up (10 minutes)

Cool-down (10 minutes)

Squats (3 sets of 8)

Leg Press (3 sets of 10)

Lunges (3 sets of 10)

Leg Extensions (3 sets of 12)

Leg Curls (3 sets of 10)

WORKOUT

04

Day 4: Shoulders and Abs

Warm-up (10 minutes)

Cool-down (10 minutes)

Dumbbell Shoulder Press (3 sets of 8)

Lateral Raises (3 sets of 10)

Rear Delt Fly (3 sets of 12)

Plank (3 sets of 30-60 seconds)

Russian Twists (3 sets of 10)

WORKOUT

Activate Windows

Go to Settings to activate Windows.

ArogyaMitra

AI Health Coach

Dashboard

Workout Plan

Nutrition Plan

Weekly Plan

Calendar

Welcome,

Ryan Gosling

Logout

<

July 2026

>

SUN	MON	TUE	WED	THU	FRI	SAT
			1	2	3	4
5	6	7	8	9	10	11
12	13	14	15	16	17	18
19	20	21	22	23	24	25
26	27	28	29	30	31	

Saved Plans

Day 1: Chest and Triceps

2026-07-28

WORKOUT

NUTRITION

Day 2: Back and Biceps

2026-07-28

WORKOUT

NUTRITION

Day 3: Legs

2026-07-30

WORKOUT

NUTRITION

Day 4: Shoulders and Abs

2026-07-31

WORKOUT

NUTRITION

Day 5: Arms and Calves

2026-08-01

WORKOUT

NUTRITION

Activate Windows

Go to Settings to activate Windows.

✓

Day 1: Chest and Triceps

✕

2026-07-28

WORKOUT

NUTRITION

Workout Plan

Warm-up (10 minutes)

+ 5 minutes on the treadmill (jogging)

+ Dynamic stretching (arm circles, leg swings, torso twists)

Main Exercises

1. Barbell Bench Press (3 sets of 8-12 reps)

2. Incline Dumbbell Press (3 sets of 10-12 reps)

3. Cable Fly (3 sets of 12-15 reps)

4. Tricep Pushdown (3 sets of 10-12 reps)

5. Overhead Dumbbell Extension (3 sets of 12-15 reps)

Cool-down (10 minutes)

+ Static stretching (focusing on chest and triceps)

Nutrition Plan

Breakfast: Oatmeal with banana, almond milk, and walnuts (350 calories)

+ 1 cup cooked oatmeal

+ 1 medium banana

+ 1 cup almond milk

+ 1 tablespoon chopped walnuts

Lunch: Grilled chicken breast with quinoa, mixed vegetables, and avocado (550 calories)

+ 4 oz grilled chicken breast

+ 1/2 cup cooked quinoa

Activate Windows
Go to Settings to activate Windows.

Chapter 19: Testing

19.1 Backend Testing

Backend testing focused on verifying that every router responded with the correct status codes and data structures under both normal and abnormal input, using the built in Swagger documentation available at the docs endpoint as the primary manual testing tool alongside Postman.

19.2 Frontend Testing

Frontend testing involved manually walking through every user flow, including registration, login, completing the health assessment, generating a workout plan, playing an exercise video, generating a nutrition plan and chatting with AROMI, to confirm that each page rendered correctly and updated in real time.

19.3 API Testing

Each API endpoint was tested individually for correct authentication handling, correct validation error messages, and correct response formatting, ensuring that the frontend could rely on a consistent contract from the backend.

19.4 Edge Cases

Edge cases that were specifically tested included a user submitting an assessment with multiple allergies, a user requesting a plan modification while already travelling, and a user attempting to log in with an incorrect password multiple times.

19.5 Validation

Input validation was enforced at the schema level using Pydantic, which meant that invalid data such as a negative age or an unsupported diet preference value was rejected before it ever reached the business logic or the database.

Chapter 20: Performance

20.1 Optimisation

Performance optimisation focused primarily on reducing the time taken to generate AI powered content, since this is the most computationally expensive part of the system.

20.2 Caching

Frequently accessed but rarely changing data, such as a user's own profile information, is cached on the frontend using React Query, reducing the number of repeated network calls made while navigating between pages.

20.3 Prompt Optimisation

Prompts sent to the Groq model were refined over multiple iterations to be concise yet complete, which reduces both the response time and the token cost of every AI request while still preserving the quality of the generated plan.

20.4 Database Queries

Database queries were structured to retrieve related data, such as a workout plan along with its exercises, in as few queries as possible, avoiding repeated round trips to the database for each individual exercise.

20.5 Lazy Loading

Frontend components that are not immediately visible, such as the detailed progress charts, are loaded lazily, which keeps the initial page load fast even as more features are added to the dashboard.

20.6 Scalability

The modular backend architecture, with clearly separated routers and services, means that individual modules such as the nutrition engine or the AROMI coach can be scaled or optimised independently as user load increases in the future.

Chapter 21: Deployment

21.1 Frontend Deployment

The React frontend is built using Vite and can be deployed as a static build to any modern hosting service, with the `VITE_API_BASE_URL` environment variable configured to point to the deployed backend address.

21.2 Backend Deployment

The FastAPI backend is served using Uvicorn and can be deployed behind a production ready process manager, with all sensitive keys such as the Groq, YouTube, Spoonacular and Google Calendar credentials supplied through environment variables rather than hard coded values.

21.3 Database Deployment

For local development, SQLite is used for its simplicity, but the SQLAlchemy based architecture allows the `DATABASE_URL` variable to be pointed at a more robust production database without requiring any change to the application code.

21.4 Environment Variables

All configuration, including database URLs, secret keys, CORS origins and third party API keys, is managed through a single `.env` file on the backend and a corresponding `.env` file on the frontend, keeping configuration cleanly separated from code.

21.5 Production Architecture

In a full production setting, the React build would be served through a content delivery network, the FastAPI backend would run behind a reverse proxy with HTTPS enabled, and the database would be hosted on a managed, backed up relational database service, ensuring the platform remains reliable as it scales beyond local development.

Chapter 22: Conclusion

22.1 Achievements

ArogyaMitra successfully brings together artificial intelligence, personalised fitness planning and modern wellness principles into a single, unified platform. By integrating Groq's LLaMA-3.3-70B model, the system is able to generate tailored workout routines, nutrition plans and real time adaptive coaching that respond dynamically to each user's lifestyle, preferences and health conditions. The platform's modular architecture, built with FastAPI, React, SQLAlchemy, the YouTube API, Spoonacular and Google Calendar, creates a smooth and interactive end to end experience where a user can complete a health assessment, receive a full week of guidance, follow along with embedded exercise videos, and track their progress, all within one connected system.

22.2 Limitations

The current version of the platform relies on the conversation history within a single session for context and does not yet retain a long term memory of a user's patterns across weeks or months. The system also depends on the continued availability of third party services such as Groq, YouTube and Spoonacular, and any disruption to these external services can temporarily reduce the richness of the generated plans. Additionally, exercise form tracking through the camera is currently based on simple motion detection rather than full pose estimation, which limits how precisely form correction feedback can be given.

22.3 Future Scope

As outlined in the future enhancements chapter, ArogyaMitra is well positioned to grow into a more complete wellness ecosystem through wearable integration, medical report scanning, voice based interaction and a persistent AI memory system. The charity based gamification model also opens the door to genuine partnerships with charitable organisations in the future, turning a purely personal fitness journey into one with measurable social impact. Overall, the project demonstrates how an intelligent, adaptive and well designed system can meaningfully support a person's long term health journey.